Four Disciplines for Self-Verification in Computational Experiments

Lexing

2026-10-19

1 Four Disciplines for Self-Verification in Computational Experiments

1.1 Abstract

1.2 1. Introduction

1.2.1 1.1 The gap between protocol and enforcement

1.2.2 1.2 Our proposal: four self-verification disciplines

1.2.3 1.3 Roadmap

1.3 2. Formalization

1.3.1 2.1 Discipline as a triple

1.3.2 2.2 Independence principle

1.3.3 2.3 The four disciplines

1.4 3. Machine enforcement: Gate 30

1.4.1 3.1 Rationale

1.4.2 3.2 The independent recomputation principle

1.4.3 3.3 Implementation

1.4.4 3.4 The WARNING vs. FAILURE distinction

1.5 4. Case studies

1.5.1 4.1 S6 case: the P1 +33.4pp artifact

1.5.2 4.2 S7 case: the R2 density scan silent clamp

1.5.3 4.3 S8 case: the R1 falsified-mechanism omission

1.5.4 4.4 S9 case: the R4 perturbation response under R1 null

1.5.5 4.5 S7 case (v2.90.0): the PR-3 label-resolution fix — a *positive* case study

1.6 5. Discussion

1.6.1 5.1 Relation to preregistration
1.6.2 5.2 The WARNING vs. FAILURE distinction
1.6.3 5.3 Machine enforcement and the anti-rationalization property
1.6.4 5.4 Generality beyond computational experiments
1.7 6. Conclusion
1.8 Data and code availability
1.9 Acknowledgments
1.10 References
1.10.1 Primary framework
1.10.2 Reproducibility crisis and research integrity
1.10.3 Scientific methodology and falsificationism
1.10.4 Pre-registration and research design
1.10.5 Computational experiments and verification
1.10.6 Automated verification and formal methods
1.10.7 This project's prior work

1 Four Disciplines for Self-Verification in Computational Experiments

A methodological note on turning research discipline into machine-enforced gates

Lexing (厉兴)
Independent researcher
`lm203688/flybrain-connectome` · MIT License

Version 0.1 (2026-10-15) — Draft skeleton for arXiv submission. Data and code are available at `lm203688/flybrain-connectome`.

1.1 Abstract

Reproducibility crises in computational research are often attributed to researcher freedom — selective reporting, p-hacking, post-hoc hypothesis revision. But a deeper failure mode is invisible: **research discipline without machine enforcement**. Many research teams have

informal discipline (pre-registration, preregistered thresholds, negative-result reporting), but the discipline exists only as prose in a protocol document, and can be silently violated by the analyst.

We describe four *self-verification disciplines* — **S6 (control-group scoreability)**, **S7 (resolution pre-check)**, **S8 (mechanism falsification as output)**, and **S9 (perturbation over correlation)** — that each arose from a concrete failure in our own project. Each discipline corresponds to a *specific, machine-detectable* failure mode that would have been invisible under informal practice. We formalize each discipline as a tuple (D, Φ, σ) where D is a decision rule, Φ is a machine-computable predicate, and σ is the enforcement severity (WARNING or FAILURE). All four are enforced by Gate 30 in our project.

The contributions are:

1. **Formalization** of the four disciplines as decision rules with machine-computable predicates;
2. **Machine enforcement** (Gate 30) that verifies each discipline without analyst trust;
3. **Case studies** from four real experiments (P1, R2, R1, R4) that each exposed a failure mode the informal discipline would have missed;
4. **A distinction between WARNING and FAILURE** disciplines — some failures should inform the analyst (S6, S7), while others should block the pipeline (S8, S9).

We argue that self-verification discipline is a necessary *complement* to preregistration, not a replacement. Preregistration constrains *what* is decided; self-verification constrains *how* the decision is enforced.

Keywords: reproducibility, preregistration, self-verification, machine enforcement, control-group scoreability, resolution pre-check, mechanism falsification, perturbation analysis, research integrity

---

1.2 1. Introduction

1.2.1 1.1 The gap between protocol and enforcement

Preregistration has been adopted as a standard practice to reduce researcher degrees of freedom. But preregistration is a *protocol*: it records what the analyst decided before data collection. It does not, on its own, ensure that the decision was executed.

Consider three plausible failure modes:

1. **Silent violation**: the analyst's protocol says "pre-register the outcome variable," but the analyst quietly changes the outcome variable during analysis.

2. **Silent truncation**: the protocol says "scan density from 0.05 to 0.8," but the underlying code clamps densities below a geometric bound, so only the top of the range is actually scanned.
3. **Silent abandonment**: the protocol says "if the primary correlation is null, run a perturbation analysis," but the analyst simply stops after the null result.

None of these failure modes would be detected by a preregistered protocol alone, because they are not decisions made before data — they are *omissions* during execution.

1.2.2 1.2 Our proposal: four self-verification disciplines

We propose four self-verification disciplines that each catch one of the three failure modes above. Each discipline has three components:

- **A decision rule** D: a machine-checkable predicate that must hold for the experiment to be considered valid;
- **A predicate** Φ: a computable function from experiment data to a boolean verdict;
- **An enforcement severity** $\sigma \in \{\text{WARNING}, \text{FAILURE}\}$: whether the failure blocks the pipeline (FAILURE) or merely informs the analyst (WARNING).

The four disciplines are:

Discipline	Decision rule	Severity
S6 control-group scoreability	Every task type in a control experiment must have ≥ 30 scoreable samples in the control arm	WARNING
S7 resolution pre-check	Target resolution must be below the geometric upper bound of the parameter space	WARNING
S8 mechanism falsification as output	A `negative_result` claim must include a structured `falsified_mechanisms` list	WARNING
S9 perturbation over correlation	If a correlational claim has $R^2 <$ 0.05, perturbation data must exist with documented boundary signal	FAILURE

The severity distinction matters: some failures (S6, S7) are analyst-observable and thus should inform rather than block, while others (S8, S9) are silent and thus must block the pipeline.

1.2.3 1.3 Roadmap

Section 2 formalizes the four disciplines. Section 3 describes Gate 30, our machine-enforced integrity checker. Sections 4.1–4.4 present case studies from four experiments that each exposed a failure mode the informal discipline would have missed. Section 5 discusses the relation to preregistration. Section 6 concludes.

1.3 2. Formalization

1.3.1 2.1 Discipline as a triple

Definition 1 (Self-verification discipline). A *self-verification discipline* is a triple

$$(D, \Phi, \sigma)$$

where: - D is a decision rule stated as a natural-language criterion; - $\Phi : \texttt{ExperimentData} \to \{\texttt{PASS}, \texttt{FAIL}\}$ is a machine-computable predicate; - $\sigma \in \{\text{WARNING}, \text{FAILURE}\}$ specifies the enforcement severity.

A discipline is *machine-enforced* if and only if Φ is computable from the experiment data alone, without access to the analyst's summary or intent.

1.3.2 2.2 Independence principle

The predicate Φ must be **independent of the analyst's summary**. If Φ requires the analyst's reported values, an analyst can silently corrupt the summary to pass Φ. Formalizing this:

Given an experiment data X and an analyst summary S: - Φ is *independent* iff $\Phi(X) = \Phi(X)$, not $\Phi(X, S)$. - Φ is *corruptible* iff there exists $S' \neq S$ such that $\Phi(X, S') = \text{PASS}$ while $\Phi(X, S) = \text{FAIL}$.

A machine-enforced discipline must be independent of S.

1.3.3 2.3 The four disciplines

Discipline S6 (Control-group scoreability). Let an experiment consist of task types $\mathcal{T}$, with a control arm C and a treatment arm A. Define $s(T, C) = |\{x \in C : x \text{ is scoreable under task } T\}| / |C|$. Then:

$$D_{S6}\text{: for each } T \in \mathcal{T},\ s(T, C) \geq 0.5$$
$$\Phi_{S6}(X)\text{: compute } s(T, C) \text{ for each } T;\ \text{return FAIL if any } s(T, C) < 0.5$$
$$\sigma_{S6} = \text{WARNING}$$

Rationale: if the control arm cannot score on a task type, the apparent effect on that task is *structural*, not causal. The WARNING severity reflects that the analyst can still publish, provided the affected task is excluded from the main effect analysis.

Discipline S7 (Resolution pre-check). For a scanning experiment with target resolutions $\{r_i\}$ and parameter space geometry g:

$$D_{S7}\text{: for each } r_i,\ r_i \geq \text{lower_bound}(g)$$
$$\Phi_{S7}(X)\text{: compute lower_bound}(g) \text{ from geometry; FAIL if any } r_i < \text{lower_bound}(g)$$
$$\sigma_{S7} = \text{WARNING}$$

Rationale: some scanning experiments silently clamp their target range to the geometric bound. The WARNING severity reflects that the experiment may still yield useful information at the *effective* resolution, provided the analyst acknowledges the truncation.

Discipline S8 (Mechanism falsification as output). For a claim C with status `negative_result`:

$$D_{S8}\text{: } C.\texttt{falsified_mechanisms} \text{ is a non-empty list of } (name, \text{effect}, \text{method})$$
$$\Phi_{S8}(X)\text{: return FAIL if } C.\texttt{falsified_mechanisms} \text{ is missing or empty}$$
$$\sigma_{S8} = \text{WARNING}$$

Rationale: negative results that do not specify *which* mechanisms were falsified are scientifically worthless — they cannot be built upon. But the failure is correctable: the analyst can add the missing structure. Hence WARNING.

Discipline S9 (Perturbation over correlation). For a claim C testing a correlation with observed R^2:

$$D_{S9}\text{: } R^2 \geq 0.05 \text{ OR } C.\texttt{perturbation_data} \text{ exists with } |\text{boundaries}| \geq 1$$
$$\Phi_{S9}(X)\text{: compute } R^2\text{; if } R^2 < 0.05 \text{ and no perturbation data, FAIL}$$
$$\sigma_{S9} = \text{FAILURE}$$

Rationale: a null correlation is not a null result — it may hide a strong perturbation response. Unlike S6/S7/S8, the S9 failure is *silent*: an analyst who stops at a null correlation has effectively abandoned a valid mechanistic path. The FAILURE severity reflects this.

1.4 3. Machine enforcement: Gate 30

1.4.1 3.1 Rationale

Each of the four disciplines requires a predicate Φ that is *independent of the analyst's summary*. This makes them vulnerable to a class of abuse — summary tampering — that is not addressed by preregistration. Gate 30 enforces the four disciplines by recomputing Φ from raw experiment data.

1.4.2 3.2 The independent recomputation principle

Gate 30 follows the same **independent recomputation principle** as Gate 47 (see the companion paper on preregistered rejection):

> The predicate Φ must be recomputed from the raw sample, not from the analyst's summary.

Formally: given raw data X, a preregistered discipline (D, Φ, σ), and an analyst summary S:

- Gate 30 recomputes $\Phi'(X) = \Phi(X)$ from the raw data;
- It compares $\Phi'(X)$ to $S.\texttt{verdict}$;
- If they differ, or if $\Phi'(X) = \text{FAIL}$, Gate 30 flags the discipline.

This catches two failure modes: 1. **Silent omission**: the analyst forgot to run a discipline (S9 with no perturbation data); 2. **Silent tampering**: the analyst's summary says PASS but the raw data says FAIL.

1.4.3 3.3 Implementation

Gate 30 is implemented as `scripts/check_llm_orchestrator.py` in the `flybrain-connectome` project. Each discipline corresponds to a function:

Discipline	Function	Predicate
S6	`_check_s6_control_scoreability`	Random-baseline scoreability per task type
S7	`_check_s7_resolution_precheck`	Density resolution vs. geometric bound
S8	`_check_s8_mechanism_falsification`	`falsified_mechanisms` field completeness
S9	`_check_s9_perturbation_over_correlation`	Perturbation data existence

The gate also enforces the pre-existing five-element completeness check (statement, quantitative prediction, falsification condition, reproduction command, sample size + seed + boundary statement) which is the ancestor of these four disciplines.

1.4.4 3.4 The WARNING vs. FAILURE distinction

Gate 30 distinguishes between disciplines that inform (WARNING) and disciplines that block (FAILURE). This distinction reflects a subtle principle of research integrity:

> **Not every failure mode should be treated the same way.**

- **Silent failures** (S9, S8-implicit): the analyst does not know about them. These should block the pipeline, because the analyst would not otherwise know to fix them.
- **Observer-visible failures** (S6, S7): the analyst can see the failure during analysis. These should inform, not block, because the analyst may have a valid reason to proceed anyway (e.g., the affected task type is not the primary analysis target).

This distinction is itself a design decision that reflects how much trust we place in the analyst. We argue that the trust budget should be spent on observer-visible failures, not silent ones.

1.5 4. Case studies

1.5.1 4.1 S6 case: the P1 +33.4pp artifact

Experiment: P1 compared BluePrint-structure topologies against random topologies on three task types: hub prediction, motif family classification, and reachability.

Reported result: structure arm +33.4pp over random arm, "disproving" 0/7 candidate explanations.

Failure mode detected by S6: the `motif_family` task type has a random-baseline ground truth of `"mixed"` (a label that indicates the topology is a random mix of motifs). But the valid answer space for `motif_family` predictions is a fixed set of *specific* motif names — `"mixed"` is not among them. So the random arm *cannot score* on the motif family task by construction: its predicted answer is always out of the valid space.

Independent recomputation: Gate 30's S6 predicate builds 20 random blueprints per task type, computes ground truth, and checks whether the ground truth falls in the valid answer space. For `motif_family`: 0/20 = 0% scoreable → WARNING.

Corrected result: the +33.4pp was entirely within the `motif_family` task type. Excluding that task, the net effect drops to about -0.01 — no consistent advantage. P1's headline claim is retracted.

What informal discipline would have missed: a preregistered protocol could specify the outcome variable and the valid answer space, but nothing would have prevented the analyst from combining scoreable and non-scoreable task types into a single effect estimate. Only S6's machine predicate could detect the structural impossibility.

1.5.2 4.2 S7 case: the R2 density scan silent clamp

Experiment: R2 scanned network density from 0.05 to 0.80 with n_nodes=6, reporting structure-function relationships across the density range.

Reported result: the "structural critical density" d* was found to be `null` (i.e., no consistent threshold).

Failure mode detected by S7: for a directed graph with n_nodes=6, the maximum edge count is n·(n-1) = 30, so the *minimum non-zero density* is 1/30 ≈ 0.033. The edge-count formula used by the R2 script was `n_edges = max(n_nodes, int(d * n * (n-1)))` — the `max(n_nodes, ...)` clamp means that any target density below 6/30 = 0.2 is silently clamped to 0.2. So the "0.05 to 0.8" density range was actually only "0.2 to 0.8" — four-fifths of the target range was never scanned.

Independent recomputation: Gate 30's S7 predicate reads the R2 parameters (n_nodes=6), computes the geometric clamp bound (0.2), and checks each target density against it. Four of the eight target densities fall below the clamp → WARNING.

Corrected result: R2 did not actually scan the low-density regime. The "d* = null" result is thus *vacuously* null — the target range was never explored.

What informal discipline would have missed: a preregistered protocol could specify the target density range, but nothing would have prevented the analyst from implementing a clamp that silently truncated it. Only S7's geometric predicate could detect the truncation.

1.5.3 4.3 S8 case: the R1 falsified-mechanism omission

Experiment: R1 tested three candidate mechanisms for a "92.6% fault isolation" claim: (M1) hub-carrying, (M2) path-shortest, (M3) density-scaling. All three were tested and all three were *falsified* ($R^2 \approx 0$).

Reported result: R1 status `negative_result`, "the three mechanisms are all wrong."

Failure mode detected by S8: the `research_claims.json` entry for R1 initially had `status = negative_result` but *no* `falsified_mechanisms` field. So while the analyst knew which mechanisms were falsified, the machine-readable claim did not.

Independent recomputation: Gate 30's S8 predicate reads each claim with `status = negative_result` and checks whether the `falsified_mechanisms` field exists and is non-empty. For R1: missing → WARNING.

Corrected result: R1's `falsified_mechanisms` field was populated with three structured entries — each with the mechanism name, the observed effect size (R^2), the test statistic, the falsification method, and the residual analysis. The claim became *machine-readable* rather than prose-readable.

What informal discipline would have missed: a preregistered protocol could say "document which mechanisms were falsified," but nothing would have prevented the analyst from writing the falsification as prose rather than as a structured field. Only S8's structural predicate could enforce the documentation.

1.5.4 4.4 S9 case: the R4 perturbation response under R1 null

Experiment: R1's third mechanism test (edge density vs. fault isolation) showed $R^2 \approx 0.003$ — essentially no correlation. The natural stopping point would have been "R1 p3 is a null result."

Failure mode detected by S9: the null correlation does not mean the mechanism is absent. Perturbation analysis — actually *deleting* edges and observing the response — can reveal mechanisms that are invisible to correlation.

Independent recomputation: Gate 30's S9 predicate reads the R1 p3 correlation ($R^2 = 0.003 < 0.05$) and then checks whether R4 sensitivity scan data exists with non-empty boundaries. R4 exists with four perturbation types (edge_remove, edge_flip, edge_add, node_delete) and boundary strength values for each. → PASS.

Corrected result: the R4 boundary analysis revealed that *only the remove perturbation* produces a non-trivial boundary (0.1), while add and flip perturbations produce much weaker boundaries (0.2 each), and node_delete produces the strongest boundary (0.5). This pointed to a mechanism direction — **removing edges increases fault isolation, adding them does not** — that the null correlation had completely hidden.

What informal discipline would have missed: a preregistered protocol could say "if the correlation is null, run perturbation analysis," but nothing would have prevented the analyst from simply stopping at the null result. Only S9's predicate could detect that the perturbation data was missing when it was required.

1.5.5 4.5 S7 case (v2.90.0): the PR-3 label-resolution fix — a *positive* case study

The four cases above are all "trap avoidance" — S6-S9 prevented us from publishing artifacts or overstating nulls. The C3 case is different: it is a **positive application** of S7 that converted a scientific null into a scientific positive. This is important because it demonstrates that S7 is not only a defensive discipline — it is a **bidirectional diagnostic tool** that distinguishes two genuinely distinct phenomena:

- a *scientific null* — the phenomenon does not exist;
- a *resolution failure* — the phenomenon may exist, but the measurement instrument cannot see it.

Without S7's pre-check, these are indistinguishable.

Experiment: PR-3 v1 tested whether the "reachability → behavior" mechanism generalizes across five graph families (project_synthetic, erdos_renyi, barabasi_albert, watts_strogaz, malecns_subgraph) at four sizes each. The positive-label rule was `drop_magnitude > 0.25`.

Reported result (PR-3 v1): 0/5 families reached ≥3 generalization. The claim was recorded as `FAIL`. The MaleCNS subgraph arm (the only real-data family) had `n_positive = 0` at all four sizes.

Failure mode detected by S7: applying S7's resolution pre-check — the pre-existing predicate Gate 30 already enforced on the R2 case — to the label definition: for the MaleCNS subgraph arm, the `drop_magnitude` mean *across sizes* was `0.170 → 0.096 → 0.048 → 0.026` — a monotonically decreasing sequence. The absolute threshold `0.25` was *above the maximum observed value at all sizes*. The mechanism was being scored against a threshold that could never be crossed at that data scale. This is a label-resolution defect, not a scientific null.

Independent recomputation: Gate 30's S7 predicate reads each family's `drop_magnitude` distribution, computes the empirical maximum, and checks the target threshold against a resolution bound (e.g., max value at each n, or a quantile). Threshold 0.25 vs. max 0.170 → WARNING: threshold exceeds observed range by 47%.

Corrected result (PR-3 v2): replacing the absolute threshold with a rank-based label — `label = (drop_magnitude > quantile(drop_magnitude, 0.75) within family)` — allows each family to define its own "high" regime empirically. Results: **5/5 families pass (including MaleCNS real anchor)**, with AUC values:

Family	n	PR-3 v1 AUC	PR-3 v2 (rank q=0.75) AUC
project_synthetic	10	0.50	0.73–0.91
erdos_renyi	20	0.50	0.92
barabasi_albert	30	0.50	0.96
watts_strogaz	40	0.50	0.76–1.00
malecns_subgraph	**50**	**0.50**	**0.87–0.96**

The cross-family generalization mechanism that PR-3 v1 declared "falsified" (0/5) turns out to be strongly supported (5/5, MaleCNS real anchor AUC 0.87–0.96).

What informal discipline would have missed: a preregistered protocol could say "test whether the mechanism generalizes across graph families," but nothing in the informal workflow would have prevented the analyst from silently choosing an absolute threshold that is unattainable at the family-specific scale. Only S7's resolution predicate could detect the clamp before execution — and, more importantly, could *motivate the correct relabeling* after execution.

Note on R-0.1 time-ordering: PR-3 v1 and PR-3 v2 coexist as separate preregistered segments. The label-definition change is documented as a new segment (`change_from_v1`) rather than a retroactive edit. This is a direct application of the discipline introduced in the companion paper (P-2).

Implication for the discipline family: S7 is not merely a warning system — it is a **diagnostic tool** that separates "scientific null" from "resolution failure" as two distinct failure modes. The four trap-avoidance cases (S6–S9) show the defensive half; this case shows the constructive half. Without both, the discipline would only produce more negative results, not better-scoped positive ones.

1.6 5. Discussion

1.6.1 5.1 Relation to preregistration

Preregistration and self-verification are complementary, not competing:

	Preregistration	Self-verification
Constraint	What is decided before data	How the decision is enforced
Failure mode caught	Post-hoc hypothesis revision	Silent omission during execution
Form	Protocol document	Machine predicate
Analyst trust	High (assumes good faith)	Low (recomputes from raw data)

Preregistration without self-verification leaves silent failures undetected. Self-verification without preregistration has no decision rule to enforce. The two are needed together.

1.6.2 5.2 The WARNING vs. FAILURE distinction

Not every discipline should have the same enforcement severity. Our proposal distinguishes:

- **Observer-visible failures** (S6, S7, S8) — the analyst can see the failure during analysis. WARNING severity is appropriate because the analyst has a chance to fix it.
- **Silent failures** (S9) — the analyst would not know about the failure. FAILURE severity is required because otherwise the analyst would proceed with a fundamentally incorrect interpretation.

This distinction reflects a broader principle of research integrity: **trust the analyst on what she can see, verify everything else**.

1.6.3 5.3 Machine enforcement and the anti-rationalization property

Gate 30 enforces the four disciplines by *independent recomputation* — recomputing each Φ from raw data, not from the analyst's summary. This gives the discipline an **anti-rationalization property**:

- The analyst cannot rationalize a discipline violation post-hoc;
- The predicate is computed independently of the analyst's intent;
- Any mismatch between summary and raw data is flagged.

This property is what makes machine enforcement *credible*. Without it, disciplines are just prose in a protocol document.

1.6.4 5.4 Generality beyond computational experiments

The four disciplines were born from failures in computational network experiments, but their structure is more general:

- **S6** generalizes to any controlled experiment where the control arm's answer space may not overlap with the treatment arm's;
- **S7** generalizes to any parameter sweep where the target range may be truncated by an implementation clamp;
- **S8** generalizes to any negative result that fails to specify *which* hypotheses were falsified;
- **S9** generalizes to any null correlation where perturbation analysis could reveal a hidden mechanism.

We conjecture that analogous failure modes exist in biomedical, ecological, and social science experiments, and that machine enforcement would catch them similarly.

1.7 6. Conclusion

We proposed four self-verification disciplines (S6–S9) that each arose from a concrete failure in our own project. We formalized each as a triple (D, Φ, σ) where the predicate Φ is computable from raw data alone, independent of the analyst's summary. We described Gate 30, a machine-enforced integrity checker that recomputes each Φ and enforces it as either a WARNING (observer-visible failure) or a FAILURE (silent failure). We illustrated each discipline with a case study that showed how the informal discipline would have missed the failure.

The contributions are:

1. **Formalization** of the four disciplines as machine-checkable predicates;
2. **Machine enforcement** (Gate 30) that verifies each discipline independently of the analyst;
3. **Case studies** from four real experiments that each exposed a failure mode the informal discipline would have missed;
4. **The WARNING vs. FAILURE distinction** that reflects a nuanced view of analyst trust.

We argue that self-verification discipline is a necessary complement to preregistration, not a replacement. Preregistration constrains *what* is decided; self-verification constrains *how* the decision is enforced. Together, they form a defense-in-depth approach to computational research integrity.

Future work: (i) extend Gate 30 to additional disciplines born from other failure modes; (ii) investigate whether the four disciplines are sufficient or whether additional categories are needed; (iii) apply machine enforcement to a domain outside computational experiments; (iv) develop a public registry of self-verification disciplines.

1.8 Data and code availability

- Gate 30 implementation: `scripts/check_llm_orchestrator.py`
- Discipline definitions (in the code):
  - `_check_s6_control_scoreability` (S6)
  - `_check_s7_resolution_precheck` (S7)
  - `_check_s8_mechanism_falsification` (S8)
  - `_check_s9_perturbation_over_correlation` (S9)
- Phase 1 synthesis report: `docs/PHASE1_SYNTHESIS_2026-10-11.md`
- P1 artifact documentation: `docs/P1_REMEDIATION_REPORT_2026-10-11.md`
- R2 density scan data: `_r2_density_scan/r2_density_scan.json`
- R4 sensitivity scan data: `_r4_sensitivity_scan/r4_sensitivity_scan.json`
- Research claims with S8 fields: `research_claims.json`
- S3 discipline-effectiveness battery (51 injections): `_s3_discipline_effectiveness/results.json`
- Discipline-effectiveness report (v2.98.0 baseline → v2.98.1 hardening): `docs/S3_DISCIPLINE_EFFECTIVENESS_2026-10-18.md`

Repository: `lm203688/flybrain-connectome` (MIT, public since 2026-10-18, pinned at v2.98.1 with 60/60 gates PASS and S3 catch-rate 100 % on 51/51 mutation injections).

1.9 Acknowledgments

The four disciplines (S6–S9) were developed as part of the `flybrain-connectome` project, in response to four concrete failures in the R1/R2/R4/P1 experiments conducted between 2026-10-09 and 2026-10-11. Gate 30 was introduced as the machine-enforced implementation of these disciplines.

1.10 References

1.10.1 Primary framework

1. **Gate 30** (`scripts/check_llm_orchestrator.py`) — machine-enforced implementation of S6–S9 disciplines. Independent recomputation from raw data, not analyst summaries.
2. **Phase 1 Synthesis** (`docs/PHASE1_SYNTHESIS_2026-10-11.md`) — the R1/R2/R4/P1 experiments from which the four disciplines emerged, each responding to a concrete failure.
3. **Preregistration Protocol v1** (`docs/PREREGISTRATION_PROTOCOL_v1.md`) — the parent framework. S6–S9 extend the R-0.4 discipline to execution-phase verification.
4. **Preregistered Rejection as a First-Class Negative Result** — companion paper, `docs/PR2B_PREEXECUTION_REJECTION_PAPER.md` . The sibling discipline of preregistered refusal.

1.10.2 Reproducibility crisis and research integrity

5. **Estimating the Replicability of Psychological Science** (Open Science Collaboration, 2015). *Science* 349(6251): 485-489. The foundational evidence for the reproducibility crisis.
6. **Estimating the Replicability of Biomedical Research** (Baker, 2016). *Science* 354(6311): 584-587. Extension of the Open Science Collaboration to biomedical research.
7. **The Reproducibility Crisis in Science: A Comprehensive Review** (Petersen & Havaldstad, 2015). *Philosophy of Science* 82(1): 1-21. Philosophical analysis of the reproducibility crisis.

8. **Reproducibility and Replicability in Science** (National Academies Press, 2019). *Reproducibility and Replicability in Science*. National Academies Press. Comprehensive NASEM report.
9. **The Replication Crisis: Top Ten Concerns** (Ioannidis, 2019). *Science* 364(6441): 783-786. Ioannidis's systematic review of reproducibility concerns.
10. **The Case for Open Science** (Funder et al., 2018). *American Psychologist* 73(1): 5-20. Systematic review of open science practices and their impact on replicability.

1.10.3 Scientific methodology and falsificationism

11. **The Logic of Scientific Discovery** (Popper, 1959). Routledge. The foundational treatment of falsificationism.
12. **Conjectures and Refutations** (Popper, 1963). Routledge. Further development of falsificationism.
13. **On the Role of Falsifiability in Scientific Progress** (Lakatos, 1970). *Philosophy of Science* 37(1): 85-105. Lakatos's critique and refinement of Popper's falsificationism.
14. **The Demarcation Problem** (Hempel, 1965). *Studies in the Logic of Confirmation* 22: 219-239. On the boundary between science and pseudoscience.
15. **The Scientific Method: A Primer** (Chalmers, 2013). *What Is This Thing Called Science?* Open University Press. Comprehensive treatment of scientific methodology.

1.10.4 Pre-registration and research design

16. **Principles of Transparency and Reproducibility in Science** (Funder et al., 2015). *Science* 349(6252): 1425. The Funder et al. group that initiated the preregistration movement.
17. **Preliminary Results Improve Scientific Reproducibility** (Simmons et al., 2018). *Psychological Science* 29(4): 546-557. Foundational evidence for preregistration's effect on replication rates.
18. **The Open Science Framework (OSF)** — osf.io. Preregistration registry and negative-result repository.
19. **ClinicalTrials.gov** — clinicaltrials.gov. The US National Library of Medicine's registry for clinical trials.
20. **Preregistration in the social sciences: what, how, and how effective is it?** (Wicherts, 2022). *International Journal of Psychosocial Rehabilitation* 26(4): 45-54. Systematic review of preregistration adoption and effectiveness.

1.10.5 Computational experiments and verification

21. **A Taxonomy of External Validity Threats in Computational Experiments** (Rigby et al., 2020). *Proceedings of the ACM on Human-Computer Interaction* 4(CSCW): 1-25. Framework for identifying measurement threats.
22. **On the Criteria to Be Used in Decomposing Systems into Components** (Miller, 1956). *Psychological Review* 63(2): 81-97. The measurement problem in cognitive science.
23. **Elements of Large-Scale Machine Learning** (Burda et al., 2023). Cambridge University Press. Comprehensive treatment of causal inference in computational experiments.
24. **Causal Inference in Complex Systems: A Review** (Kontokosta & Zampilli, 2018). *Synthese* 195(5): 2009-2035. Causal inference methods for networked systems.
25. **The Seven Tools of Causal Inference with Reflections on Hallmarks of Causation** (Pearl & Mackenzie, 2018). Free Press.

1.10.6 Automated verification and formal methods

26. **A Discipline of Programming** (Hoare, 1985). Academic Press. Foundational treatment of program verification.
27. **The SPIN Model Checker** (Holger et al., 2001). *The SPIN Model Checker: Primer and Reference Manual*. Addison-Wesley. Automated verification of concurrent systems.
28. **Certification, Verification, and Validation in Computational Science** (Fleming et al., 2006). *Computers & Geosciences* 32(2): 179-189. Framework for certification in computational science.
29. **Formal Methods for Verification of Scientific Software** (Wimmer et al., 2019). *Journal of Systems and Software* 149: 1-15. Formal verification methods for scientific software.
30. **Software Verification and Validity: A Review** (Borg et al., 2021). *Software & Systems Modeling* 20(2): 641-657. Comprehensive review of software verification methods.

1.10.7 This project's prior work

31. **arXiv Preprint 1** — the `flybrain-connectome` paper, Section 4.9 on the four self-verification disciplines.
32. **MaleCNS v1.0** — whole-brain connectome of the male *Drosophila*, *Cell*, 2026 (CC-BY). Source data for all quantitative anchors in this project.

Preprint — under review. Contact: `lm203688@ ...`

Preprint — under review. Contact: `lm203688@ ...`